

Incident Report — Acme Financial Services

Author: Necdet Şanlı | Incident date: 15 Oct 2024

Time standard: All sources log in the same timezone; for this report we treat it as UTC.

Section 1: Incident Analysis

1.1 Timeline reconstruction (UTC)

Time (UTC)	Source	IP / UA	Target	Event	Evidence
06:45	API	203.0.113.45 (Acme-Mobile-Android/3.2.0)	/api/v1/login	Valid authentication ; JWT issued/used → 200	api_logs.csv
06:46	API	203.0.113.45 (Acme-Mobile-Android/3.2.0)	/api/v1/portfolio/1523	Owner read by 1523 → 200 (token verified)	api_logs.csv
06:47	API	203.0.113.45 (Acme-Mobile-Android/3.2.0)	/api/v1/portfolio/1524...1538	BOLA/IDOR: token for 1523 reads foreign accounts 1524..1538 → 200 (≥15 accounts)	api_logs.csv
~06:47	WAF	203.0.113.45	/api/v1/portfolio/15xx	Rapid Sequential Access / Possible Account Enumeration signatures → DETECT (not blocked)	waf_logs.csv
09:00	Email	203.0.113.45/ security@acme-finance.com	Employees	Phishing emails sent; sender domain differs; external sending IP; User1, User3, User5 clicked	email_logs.csv

09:20	WAF/Web	203.0.113.45 (Windows / Chrome)	/dashboard/se arch	SQL-shaped input ticker=AAPL' OR 1=1— flagged and blocked → 403	waf_logs.csv, web_logs.csv
09:21 - 09:22	WAF/Web	203.0.113.45 (Windows / Chrome)	/dashboard/se arch	DROP TABLE / UNION SELECT flagged and blocked → 403	waf_logs.csv, web_logs.csv
09:23	WAF/Web	203.0.113.45 (Windows / Chrome)	/dashboard/se arch	Bypass with MySQL versioned comment “/*!50000OR* / 1=1—” → 200 , large response (~156,789 B)	waf_logs.csv, web_logs.csv
09:24	WAF/Web	203.0.113.45 (Windows / Chrome)	/dashboard/ex port?format=c sv	Probable exfiltration: 200 , very large response (~892,341 B)	waf_logs.csv, web_logs.csv
Context	Notes	203.0.113.43 (vendor ASN)	-	P belongs to CyberSec Partners; authorized pen-test window 20–25 Oct, events are 15 Oct	security_test_ schedule.pdf

1.2 Attack vector identification

Evidence supports three concurrent vectors on the same day.

- API (BOLA/IDOR). /api/v1/portfolio/{account_id} returns 200 for non-owner IDs when a valid JWT for 1523 is presented; the service authenticates tokens but does not enforce object ownership (api_logs.csv, api_docs.pdf).
- Web (SQL injection). Signature-based defenses block some payloads, but a MySQL versioned-comment tautology is accepted, returning a large result set and followed immediately by CSV export, which is a strong exfiltration pattern (web_logs.csv, waf_logs.csv)

- Email (phishing). Messages sent later the same day from an external sender/domain; multiple clicks recorded (User1, User3, User5). Treated as a separate vector because it occurs after the JWT activity (email_logs.csv).

1.3 Attack classification (MITRE ATT&CK and OWASP)

- **MITRE ATT&CK: T1078 Valid Accounts** (reused JWT); **T1190 Exploit Public-Facing App** (SQLi); **TA0009/TA0010 Collection/Exfiltration** (API reads; CSV export); **T1566 Phishing** (later same day).
- **OWASP: API1:2019 Broken Object Level Authorization (BOLA)**; **A03:2021 Injection**; **A09:2021 Security Logging & Monitoring Failures** (WAF **DETECT** without block for enumeration).

1.4 Root cause analysis

The **primary root cause** is **missing object-ownership enforcement** on `/api/v1/portfolio/{account_id}`. Tokens are validated, but the system does **not** require `JWT.sub == account_id`, enabling cross-tenant reads (BOLA/IDOR). This aligns with the API documentation's ownership caveat (api_docs.pdf).

A **second root cause** is **insufficient positive validation/parameterization** in the web tier combined with **signature-based WAF** operating in mixed **DETECT/BLOCK** mode. The **versioned-comment** payload bypassed controls and produced a large response; this was followed by a large **CSV export**. The combination indicates **data exposure** at the web layer.

Contributing factors: relaxed rate-limiting on sensitive endpoints, lack of DB-level **Row-Level Security (RLS)/ABAC**, and detections that were not tied to immediate containment.

1.5 Impact assessment

API confidentiality. Unauthorized read access to at least **15** portfolios (IDs **1524–1538**) with a token belonging to **1523**. No evidence of writes/modification.

Web confidentiality. The **large 200** on `/dashboard/search` followed by a **very large 200** on `/dashboard/export?format=csv` within one minute suggests **probable data exfiltration**. Final scope requires sampling **response bodies**, **DB query logs**, and **export job metadata**.

Attribution context. Traffic originates from **203.0.113.45**, within the vendor's ASN but **outside** the authorized pen-test window. Treat as **malicious**, not sanctioned.

Phishing. Observed later; relevant to enterprise risk but **not shown** to be the source of the earlier JWT in this incident.

Section 2: Architecture Review

2.1 Current architecture weaknesses

Authorization is **incomplete**: portfolio reads lack **ownership checks**; web search accepts untrusted inputs without **allow-listing** and can return **large result sets**; WAF is **signature-centric** with inconsistent blocking; the data layer lacks **RLS/ABAC** to bind rows to identities; and there are no high-fidelity detections that automatically contain **enumeration** or **SQL-pattern + 200 (large)** events.

2.2 Improved security architecture (overview)

- **Edge (WAF):** Enforce strict positive validation on `/search` and `/export`; block tautologies, `UNION`, and MySQL versioned comments; normalize inputs; throttle sequential ID probes. (OWASP: **A03:2021 Injection**, **ASVS V5 Validation**, **API4:2023 Lack of Rate Limiting**; MITRE: **T1190 Exploit Public-Facing Application**)
- **API Gateway (policy choke point):** Mandatory ownership guard (`JWT.sub == account_id` → **403**), per-route rate limits/burst control, signed decision logs. (OWASP: **API1:2023 Broken Object Level Authorization**, **ASVS V4 Access Control**, **API4 Rate Limiting**; MITRE: **T1078 Valid Accounts**—abuse prevention via tighter authz)
- **Application (Portfolio Service):** Enforce ABAC/RLS semantics in code (subject–resource binding), **parameterized queries** only, read-only roles for GET, cap fields/rows and require signed export jobs. (OWASP: **API1 BOLA**, **API3 Excessive Data Exposure**, **A03 Injection**, **ASVS V4/V5**; MITRE: **T1190** for injection prevention, **T1020 Automated Exfiltration**—export controls)
- **Database (hard backstop): Row-Level Security** (`owner_id == sub`) regardless of app logic; separate read/write roles; alert on abnormal row counts/long reads. (OWASP: **API1 BOLA**, **ASVS V4 Authorization**; MITRE: **T1078** containment, **T1020**/exfil reduction)
- **Detection & response (SOC plane):** High-signal rules—API: `sub != account_id` & **200** (BOLA); Web: **SQL-pattern + large 200 / abnormal export**; stream WAF/GW/App to SIEM and auto-contain via playbooks (revoke tokens, block IPs, update WAF, notify). (OWASP: **API10:2023 Insufficient Logging & Monitoring**, **ASVS V7 Logging**; MITRE: **TA0009/TA0010 Collection/Exfiltration**, **T1041 Exfiltration Over C2 Channel**—detect/contain)
- **Email security (parallel control):** DMARC alignment + safe-links/sandboxing; feed detections to SIEM for campaign correlation and user-risk scoring. (OWASP: **ASVS V2/V3 Auth/Sess management** & **OOB protections**; MITRE: **T1566 Phishing**—prevention/detection)
- **Client & tokens:** Browser: **HttpOnly/Secure/SameSite** cookies, strong **CSP**, tight **CORS**, never put tokens in URLs. Mobile: **Auth Code + PKCE**, secure keystore, deep-link allowlists. (OWASP: **ASVS V2/V3 Auth/Session**, **V5 Validation**, **API8 Injection** avoidance patterns; MITRE: **T1078 Valid Accounts** hardening)

2.3 Recommended security controls

- **Gateway ownership check (403 on mismatch).** Enforce `JWT.sub == account_id` at the earliest control point; simple, auditable, and blocks BOLA immediately.
- **ABAC/RLS in service & DB.** Bind queries to identity (`owner_id == JWT.sub`) to eliminate cross-tenant reads even if an upper layer fails.
- **Strict input validation + parameterized queries.** Positive allow-lists for search (e.g., ticker symbols), remove dynamic SQL; this neutralizes SQLi regardless of WAF signatures.
- **WAF hardening & virtual patching.** Set **BLOCK** for tautologies, `UNION`, and **MySQL versioned comments**; apply targeted rules to `/dashboard/search` and

/dashboard/export.

- **Rate limits & enumeration detection.** Throttle sequential `account_id` access and alert on patterns indicative of BOLA probing.
- **SIEM/SOAR automation.** Correlate `sub=account_id & 200` and `SQL-pattern + 200 (large)`; auto-revoke tokens, quarantine IPs, trigger user notifications.
- **Email posture.** Enforce **DMARC p=reject** + alignment; use link sandboxing; sustain awareness training..

Section 3: Response & Remediation

3.1 Immediate actions (0–24 hours)

Enforce `JWT.sub == account_id` at the gateway and hot-patch service logic; **disable or restrict** /dashboard/export to **admin + MFA**; set WAF to **BLOCK** for `OR 1=1, UNION, and /*!50000...*/` patterns; deploy **allow-lists** on search inputs; **revoke** tokens/sessions tied to **203.0.113.45**; run a **90-day** retrospective for `status=200 & user_id=account_id` (API) and `SQL-pattern + 200 (large)` (Web); collect **response bodies**, **DB query logs**, and **export records** for **09:20–09:30 UTC** to quantify leaked fields/rows.

3.2 Short-term fixes (1–2 weeks)

Implement **ABAC/RLS**; migrate all queries to **parameterized** forms; add **rate limits** and **enumeration anomaly** detection; apply WAF **virtual patches** on search/export; ensure all GETs use **read-only DB roles**; cap export size and require **signed export jobs** with audit trails.

3.3 Long-term improvements (1–3 months)

Shorten token lifetimes; adopt token binding/DPoP for mobile and secure token storage with device integrity checks; enforce mTLS between tiers and rotate keys; build SIEM correlations with SOAR auto-containment; integrate DAST/negative tests for SQLi/BOLA into CI; maintain DMARC enforcement and phishing training.

3.4 Compliance considerations

Assess **KVKK/GDPR** notification thresholds based on confirmed export contents; preserve logs and artifacts for audit; update the risk register and mappings (OWASP API Security; **NIST SP 800-53 AC/IA/SC/AU/IR**; **ISO/IEC 27001 A.8/A.12.4/A.16**). If personal/regulated data was exported, initiate the organization's **data-breach** workflow and notifications.